

# Testing of a Steganographic Tool: Text-in-Image Implementation

**Student:** Irmak Yılmaz

**Student Number:** E-379

## 1. Chosen Steganography Method

For this assignment, I chose "text-in-image steganography." Specifically, the program uses the **Least Significant Bit (LSB) substitution method**. In digital imaging, a standard pixel consists of three color channels (Red, Green, Blue), each represented by 8 bits (1 byte). The LSB method works by replacing the very last bit (the least significant one) of these color bytes with the binary data of our secret text message. Because changing only the last bit alters the color value by a maximum of 1 unit (e.g., from 145 to 144), the visual difference is completely undetectable to the human eye.

## 2. Detailed Explanation of the Program

I implemented the tool using Python and the Pillow library for image manipulation.

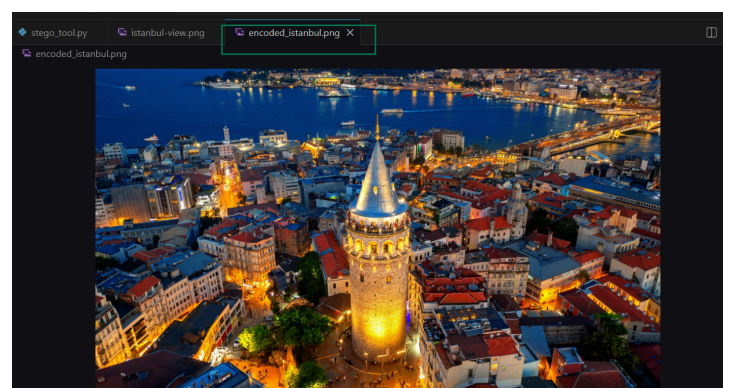
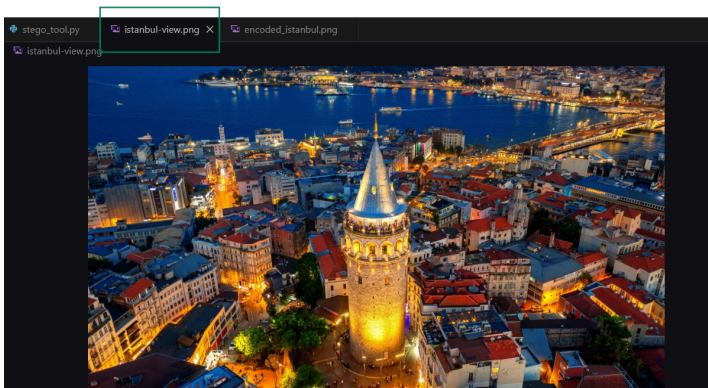
- **Hiding Information:** The program first converts the secret text message into a continuous binary string. It appends a specific delimiter (#####) to the end of the message to signal where the hidden text stops. It then iterates through the pixels of the carrier image. For each RGB channel of a pixel, it clears the existing LSB using a bitwise AND operation ( $\& \sim 1$ ) and inserts the next bit of the secret message using a bitwise OR operation ( $\mid 1$ ). Finally, it saves the modified image as a lossless PNG file.
- **Retrieving Information:** The extraction function reads the encoded image pixel by pixel. It extracts the LSB of every color channel using a bitwise AND operation ( $\& 1$ ) and reconstructs the binary string. This string is continuously converted back to ASCII characters until the program encounters the unique delimiter (#####), at which point it prints the successfully retrieved secret message.

## 3. Demonstration of the Program's Functionality

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\23022\OneDrive\Desktop\ders> python stego_tool.py
=== Steganography Tool (LSB Method) ===

Hiding message: 'Erasmus Secret Data: Greetings from an Istanbul in Poland. The server password is 'Galata2026!''
[SUCCESS] Message successfully hidden in 'encoded_istanbul.png'

Extracting message from the encoded image...
[REVEALED MESSAGE] -> Erasmus Secret Data: Greetings from an Istanbul in Poland. The server password is 'Galata2026!'
```



#### 4. Limitations of the Solution

During the development and testing of this tool, I identified several inherent limitations of the LSB steganography method:

1. **Capacity Limit:** The maximum length of the hidden message is strictly limited by the resolution of the carrier image. An image with dimensions  $W \times H$  can hold a maximum of  $(W \times H \times 3)/8$  bytes of text.
2. **Vulnerability to Compression (Lossy Formats):** The carrier file must be saved in a lossless format like PNG or BMP. If the encoded image is sent over a platform that compresses images (like WhatsApp or saving it as a JPEG), the compression algorithm will discard or modify the LSBs, completely destroying the hidden message.
3. **Resilience to Modifications:** The steganography is highly fragile. Any basic image processing operation—such as resizing, cropping, rotating, or applying color filters will alter the pixel values and corrupt the hidden data.
4. **Ease of Detection (Steganalysis):** While invisible to the human eye, LSB modifications can be easily detected by statistical analysis algorithms. A sudden increase in the entropy of the least significant bits usually reveals the presence of hidden data.